***Володимирський педагогічний фаховий коледж імені Агатангела Кримського***
циклова комісія викладачів професійної цифрової та математичної освіти

**Інструкція до практичної роботи №23**

**ТЕМА: Сортування списків (алгоритми «бульбашки» та вибору).**

*Освітньо-професійна програма: Професійна освіта (Цифрові технології)*
*Тривалість заняття: 80 хв*

**Мета заняття:**

- ознайомитися з поняттям алгоритму сортування;
- вивчити принцип роботи сортування «бульбашкою» (bubble sort);
- вивчити принцип роботи сортування вибором (selection sort);
- навчитися реалізовувати обидва алгоритми мовою Python та порівнювати їх.

## Теоретичні відомості

### *1. Навіщо потрібні алгоритми сортування*

Сортування — це впорядкування елементів списку за певним правилом (зазвичай за зростанням або спаданням значень). Хоча в Python є вбудований метод sort(), який робить це автоматично, розуміння того, як сортування працює «зсередини», є важливою базовою навичкою програміста та часто зустрічається на співбесідах і олімпіадах.

### *2. Сортування бульбашкою (bubble sort)*

Ідея методу: алгоритм багаторазово проходить списком, порівнюючи кожну пару сусідніх елементів, і міняє їх місцями, якщо вони стоять у неправильному порядку. За один прохід найбільший елемент «спливає» у кінець списку — подібно до бульбашки повітря у воді, звідки й назва методу.




*Рис. 1. Порівняння та обмін місцями сусідніх елементів*

```
numbers = [8, 5, 3, 9, 2]
n = len(numbers)
for i in range(n - 1):
    for j in range(n - 1 - i):
```

```
        if numbers[j] > numbers[j + 1]:
            numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]
print(numbers)
```

***Результат:*** *[2, 3, 5, 8, 9]*

Зверніть увагу на команду обміну значеннями numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j] — це стандартний прийом Python для обміну значень двох змінних (чи елементів списку) без потреби у додатковій тимчасовій змінній.

Зовнішній цикл (за i) визначає кількість повних проходів списком, а внутрішній (за j) виконує самі порівняння сусідніх елементів. На кожному наступному проході внутрішній цикл можна робити коротшим, оскільки останні і елементів уже гарантовано відсортовані.

### *3. Сортування вибором (selection sort)*

Ідея методу: на кожному кроці серед ще не відсортованої частини списку знаходять мінімальний елемент і переставляють його на початок цієї невідсортованої частини (тобто на чергову позицію зліва).

```
numbers = [8, 5, 3, 9, 2]
n = len(numbers)
for i in range(n - 1):
    min_index = i
    for j in range(i + 1, n):
        if numbers[j] < numbers[min_index]:
            min_index = j
    numbers[i], numbers[min_index] = numbers[min_index], numbers[i]
print(numbers)
```

***Результат:*** *[2, 3, 5, 8, 9]*

На відміну від бульбашки, де обмін може відбуватися багато разів за один прохід, сортування вибором виконує не більше одного обміну за кожен прохід зовнішнього циклу — це робить його трохи ефективнішим за кількістю фактичних перестановок, хоча кількість порівнянь у обох методів приблизно однакова.

### *4. Порівняння двох методів*

| Критерій | Бульбашка | Вибір |
|---|---|---|
| Принцип | багаторазовий обмін сусідніх елементів | пошук мінімуму і одна перестановка за прохід |

| | | |
|---|---|---|
| Кількість обмінів | може бути велика | не більше n-1 |
| Простота розуміння | дуже наочний | трохи складніший для розуміння |

## Хід роботи

### *Варіант і вимоги до здачі*

а) N — ваш порядковий номер у списку групи (журнал). Список для завдань 1–4 і 6* залежить від N (mod — остача від ділення):

| **Завдання** | **Дані варіанта** | **Приклад N = 7** |
|---|---|---|
| 1–4, 6* | 9 чисел: $a_0 = N$ (у список не входить), $a_i = (5 \cdot a_{i-1} + N) \bmod 40$, $i = 1…9$ | [2, 17, 12, 27, 22, 37, 32, 7, 2] |

б) Кожне завдання — окремий файл Прізвище_ПР23_зN.py (наприклад, Ivanenko_PR23_z3.py), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) Не використовуйте методи sort(), sorted() та функції min(), max() для самого сортування: алгоритми треба реалізувати власноруч. Обмін значень — командою a, b = b, a.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Створіть файл Прізвище_ПР23_з1.py. Реалізуйте сортування бульбашкою (за зростанням) для списку вашого варіанта (9 чисел, задайте його прямо в коді). Виведіть список до і після сортування.

   **Приклад (N = 7):** до: [2, 17, 12, 27, 22, 37, 32, 7, 2]; після: [2, 2, 7, 12, 17, 22, 27, 32, 37].

2. Реалізуйте сортування вибором для того самого списку (працюйте з його копією, щоб не зіпсувати вихідний), виведіть результат і порівняйте його з результатом бульбашки: виведіть «Результати збігаються» або «Результати відрізняються».

**Приклад:** для списку N = 7 → [2, 2, 7, 12, 17, 22, 27, 32, 37] і «Результати збігаються».

3. Додайте до обох алгоритмів лічильник обмінів і виведіть «Обмінів у бульбашці: …» та «Обмінів у виборі: …». Рахуйте лише справжні обміни: у виборі — коли індекс мінімального елемента не збігається з поточною позицією.

   **Приклад (N = 7):** «Обмінів у бульбашці: 16», «Обмінів у виборі: 6».

4. Модифікуйте сортування бульбашкою так, щоб воно сортувало список за спаданням (змініть лише знак порівняння), і виведіть результат.

   **Приклад (N = 7):** [37, 32, 27, 22, 17, 12, 7, 2, 2].

5. Запитайте кількість елементів n, а потім n цілих чисел (по одному через input()), відсортуйте їх одним із вивчених методів і виведіть результат. Якщо n = 0, виведіть «Список порожній».

   **Приклад:** n = 5, числа 5, 3, 8, 1, 9 → [1, 3, 5, 8, 9]; n = 0 → «Список порожній».

6. Завдання підвищеної складності*. Оптимізуйте сортування бульбашкою: додайте логічну змінну-прапорець, що показує, чи був хоча б один обмін під час проходу; якщо обмінів не було — завершіть зовнішній цикл командою break. Виведіть відсортований список і «Проходів: …» (прохід без обмінів теж рахується).

   **Приклад:** [1, 2, 3, 4, 5] → «Проходів: 1»; [2, 1, 3, 4, 5] → «Проходів: 2»; список N = 7 → «Проходів: 8».

## Критерії оцінювання

| Складова | Оцінювання | Макс. балів |
|---|---|---|
| Завдання 1–5 | кожне по 1,8 бала | 9 |
| Завдання 6* | 2 бали | 2 |
| Контрольні питання | відповіді у звіті | 1 |
| **Разом** | | **12** |

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, використано sort() замість алгоритму, обмін лічильником рахується неправильно, не оброблено n = 0, відсутній break у завданні 6*) — 50%; програма не працює або не здана —

0. Завдання 1–5 оцінюються по 1,8 бала, завдання 6* — 2 бали. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

***Захист роботи***

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити напрямок сортування чи розмір списку). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

## Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Опишіть своїми словами принцип роботи сортування бульбашкою.
2. Опишіть своїми словами принцип роботи сортування вибором.
3. Яка команда Python дозволяє обміняти значення двох змінних без додаткової тимчасової змінної?
4. Чим відрізняється кількість обмінів місцями у бульбашці порівняно з сортуванням вибором?
5. Як визначити, чи список уже повністю відсортований, не завершуючи всі заплановані проходи бульбашки?